



DMS624

FINAL PROJECT REPORT

Members:

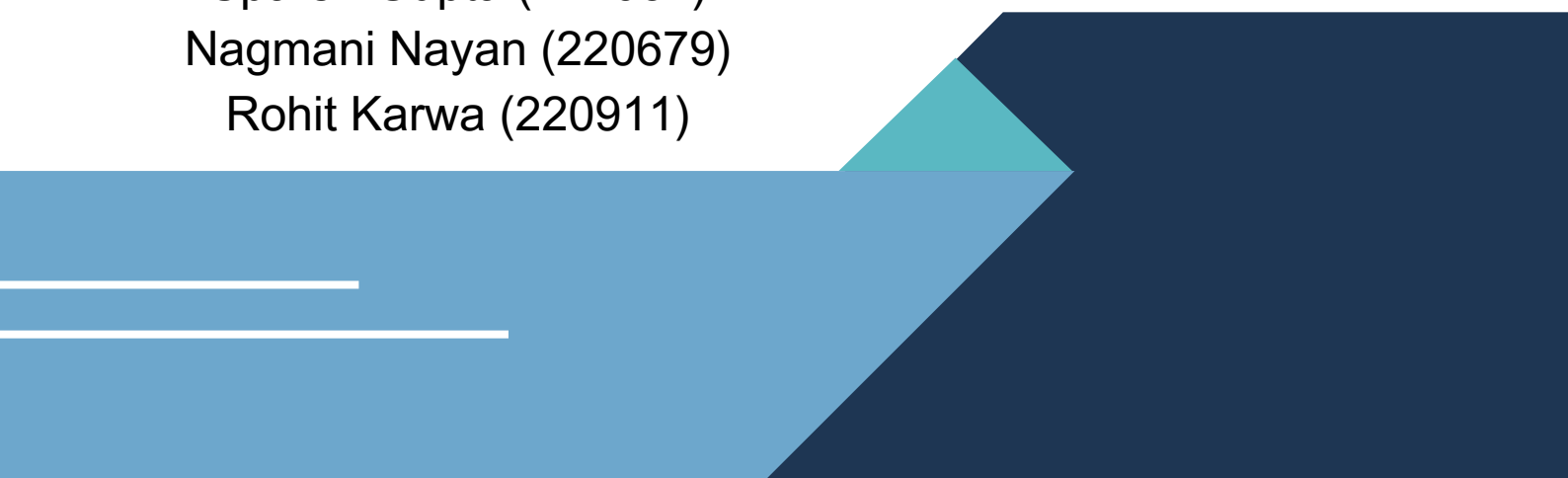
Anuradha Singh (208070169)

Vatsal Rajput (221179)

Sparsh Gupta (221084)

Nagmani Nayan (220679)

Rohit Karwa (220911)



Introduction

- The state road transport corporation (SRTC) is a critical component of public transportation infrastructure, providing connectivity across cities, towns, and rural areas. To enhance efficiency and provide better services, an automated Decision Support System (DSS) is envisioned. The first step in building this DSS is the creation of a robust, scalable, and well-structured database.
- This report outlines the design of the Entity-Relationship (ER) model, which will serve as the backbone for the database. The model incorporates various components, including corporations, routes, trips, stops, schedules, and fare systems. It ensures seamless integration of data to meet the demands of route optimization, scheduling, and cost-effective transportation management.
- The document includes detailed explanations of entities, relationships, assumptions, derived attributes, and their implementation in the database. The goal is to provide a blueprint that can be expanded to accommodate a national-level system for public transportation.

Assumptions

1) Corporate Management:

- Each corporation has a unique identity within the database and operates multiple routes. Corporations may share stops and routes for intercity or interstate connectivity.

2) Stop Characteristics:

- Stops are uniquely identified by their IDs, and multiple routes can utilize the same stop. Stops are classified into two categories: normal stops and stations, differentiated by their type value.

3) Trip Operations:

- A trip represents a single vehicle journey along a route and includes intermediate stops. Each trip is governed by a service schedule defined by a calendar and service dates.

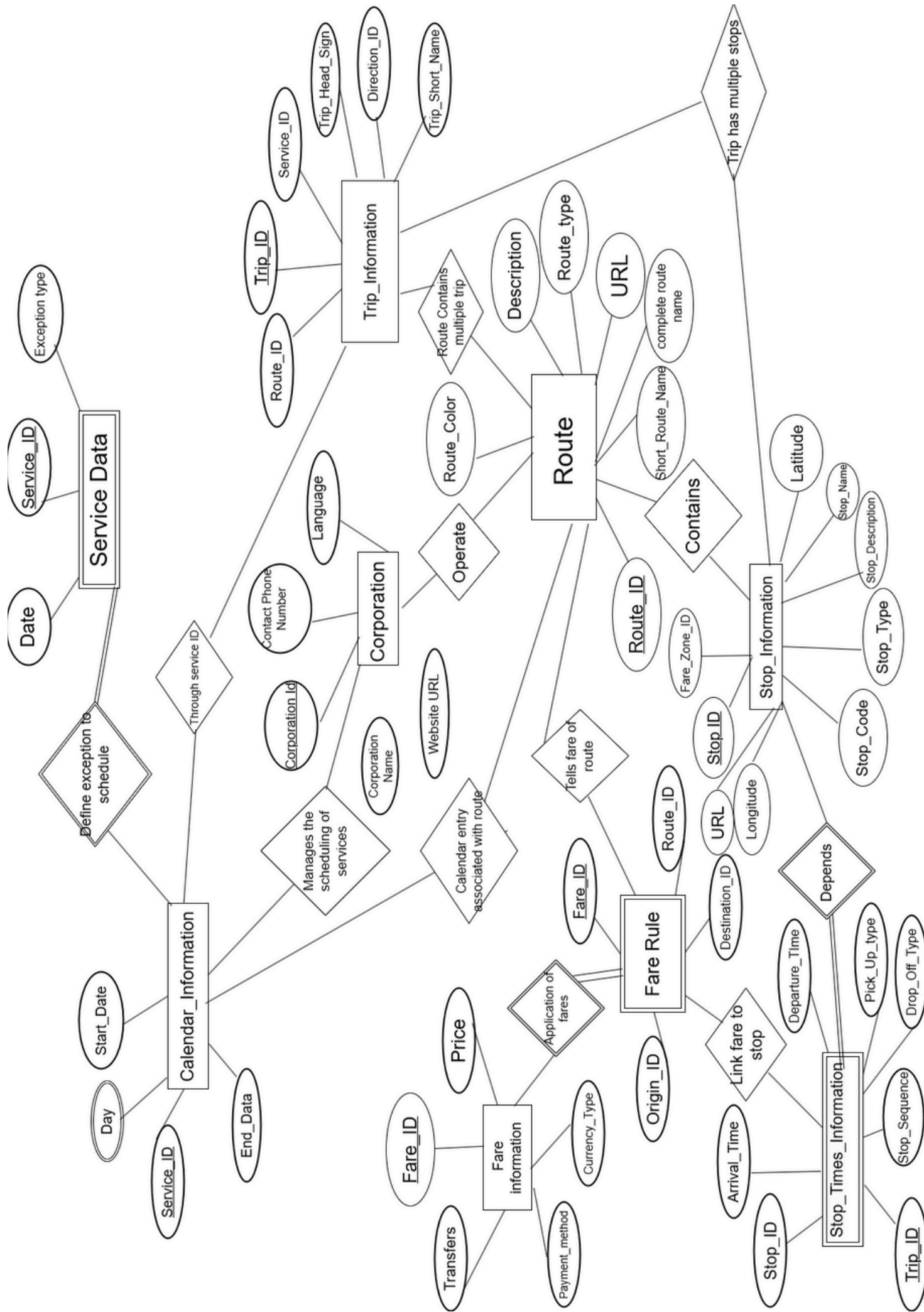
4) Schedules:

- Operational schedules are defined using binary flags for weekdays and explicit exceptions for specific dates.
- Time attributes like arrival and departure follow a standard HH:MM:SS format.

5) Fares and Rules:

- Fares depend on zones (defined by stops), routes, and payment methods.
- Transfer rules are explicitly stated, allowing for single or unlimited transfers depending on fare class.

E-R Diagram for State Road Transportation System



Dependencies between entities

The relationships between the entities are outlined below:

1) Corporation ↔ Routes

Type: One-to-Many

Explanation: Each corporation manages multiple routes, but a route is operated by a single corporation.

2) Routes ↔ Stops

Type: Many-to-Many

Explanation: Multiple routes may use the same stop, necessitating an associative entity such as Route_Stop.

3) Routes ↔ Trips

Type: One-to-Many

Explanation: Each route has multiple trips, and each trip belongs to a single route.

4) Trips ↔ Stops

Type: One-to-Many (via Stop Times)

Explanation: A trip consists of multiple stops, and stop timings are managed through the Stop Times entity.

5) Trips ↔ Calendar

Type: One-to-One

Explanation: Each trip follows a specific schedule defined by a service ID in the calendar.

6) Calendar ↔ Service Dates

Type: One-to-Many

Explanation: Service dates modify or add exceptions to the base schedule in the calendar.

7) Fare Rules ↔ Routes/Stops

Type: Many-to-Many

Explanation: Fares apply to specific routes and stops through zones, requiring an association between routes and zones.

Weak entities in our database

1. Stop Times Information

Why Weak?

The Stop Times table depends on the Stops table (via stop_id) for its identification.

Attributes like arrival_time, departure_time, and stop_sequence are meaningful only in the context of a specific trip.

Attributes:

- 1.trip_id (Foreign Key from Trips)
- 2.stop_id (Foreign Key from Stops)
- 3.arrival_time, departure_time, stop_sequence, pickup_type, drop_off_type.

Owner Entities: Stops.

2. Service Dates Information

Why Weak?

The Service Dates table depends on the Calendar table (via service_id) to define exceptions to the default calendar schedule.

It contains additional attributes (date, exception_type) that are not unique without service_id.

Attributes:

- 1.service_id (Foreign Key from Calendar)
- 2.date, exception_type.

Owner Entity: Calendar.

3. Fare Rules Information

Why Weak?

The Fare Rules table depends on the Fares, Routes. The primary key is a composite of fare_id, route_id, origin_id, and destination_id, indicating it cannot exist independently.

Attributes:

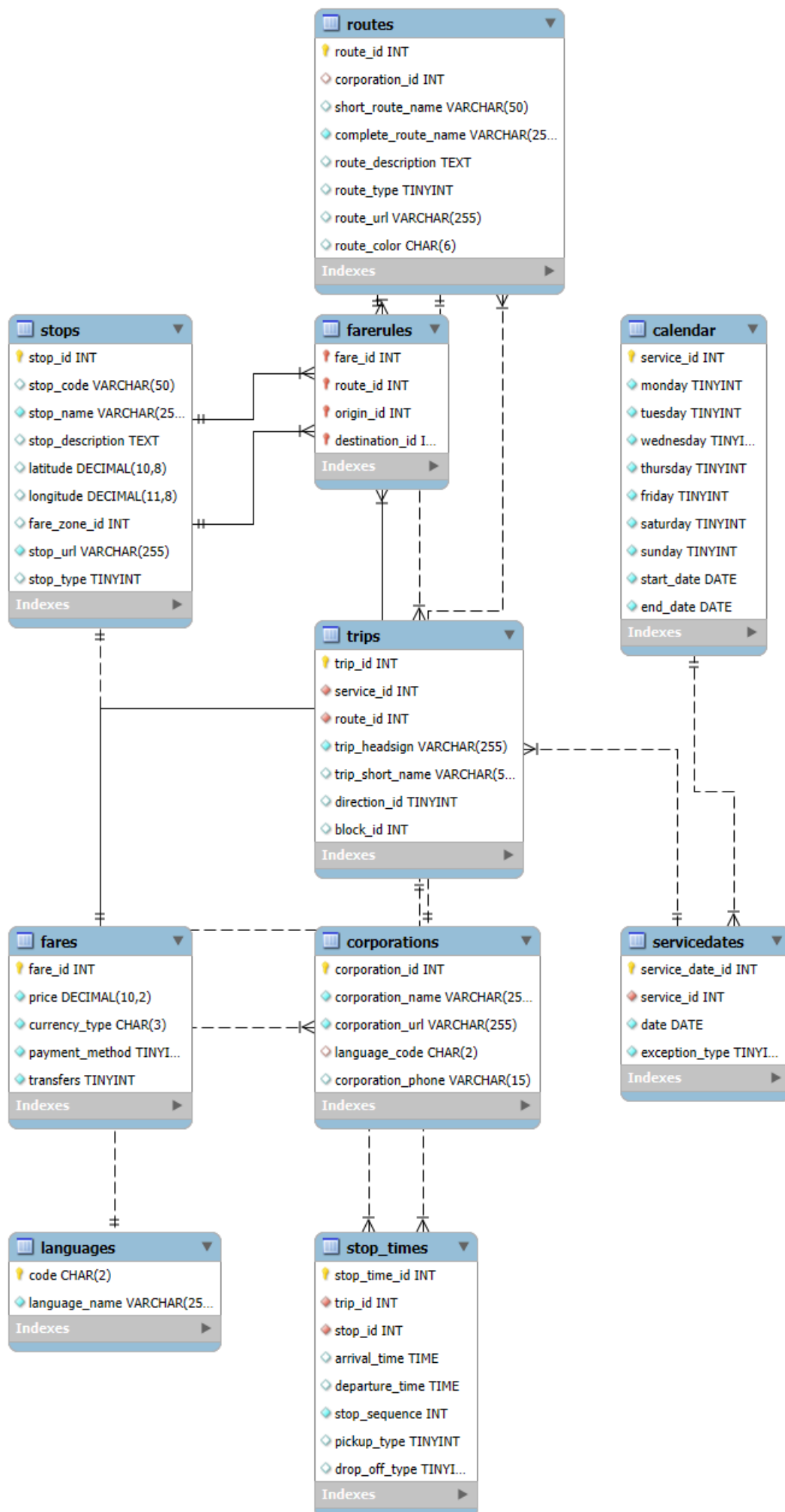
- 1.fare_id (Foreign Key from Fares)
- 2.route_id (Foreign Key from Routes)
- 3.origin_id, destination_id (Foreign Keys from Stops).

Owner Entities: Fares

Summary of Weak Entities:

Weak Entity	Owner Entities	Reason
Stop Times	Trips , Stops	Relies on trip_id and stop_id for identification.
Service Dates	Calendar	Relies on service_id for exceptions to calendar schedules.
Fare Rules	Fares , Routes , Stops	Relies on foreign keys (fare_id , route_id , origin_id , destination_id).

Data Base Schema for transport database



Field specific constraints in our database

1) Corporation Table:

- corporation id must be unique.
- corporation url must be a fully qualified URL including http:// or https://.
- language must be a valid two-letter code as mapped in a separate language table.

2) Stops Table:

- stop id must be unique.
- Latitude and longitude must conform to the WGS 84 standard.
- stop type must be either 0 (normal stop) or 1 (station).

3) Routes Table:

- route id must be unique.
- route type must be one of the predefined values (0 to 4).
- route color must be a valid six-character hexadecimal value.

4) Trips Table:

- trip id must be unique.
- direction id must be binary (0 or 1).
- block id must be referenced by at least two trips.

5) Stop Times Table:

- trip id must reference the trips table.
- stop id must reference the stops table.
- Arrival and departure times must be in HH:MM:SS format.
- Stop sequence values must be positive integers and increase sequentially along the trip.
- pickup type and drop-off type must be 0 (regular) or 1 (unavailable).

6) Calendar Table:

- Binary values for weekdays (0 or 1).
- start date and end date must be valid and in MySQL YYYYMMDD format.
- Service Dates Table:
 - (service id, date) must be unique.
 - exception type must be 1 (service added) or 2 (service removed).

7) Fare Table:

- fare id must be unique.
- currency type must follow ISO 4217 codes.
- payment method must be 0 (on board) or 1 (before boarding).
- transfers must be 0 (none), 1 (single), or 2 (unlimited).

8) Fare Rules Table:

- fare id must reference the fare table.
- route id, origin id, and destination id must reference their respective tables.

Normalisation of database

1. First Normal Form (1NF)

Requirement:

- Ensure all tables have a unique primary key.
- Eliminate duplicate columns in a table.
- Ensure each column contains atomic (indivisible) values.

Example:

If the Stops table currently has a field like Stop_Location that stores "City, State, Zip," break it into:

Stop_ID	City	State	Zip Code
001	Mumbai	Maharashtra	400001
002	Delhi	Delhi	110001

2. Second Normal Form (2NF)

Requirement:

- Remove partial dependencies (no field should depend only on part of a composite primary key).
- Every non-key field must depend entirely on the primary key.

Example: If our Routes table stores route details like this:

Route_ID	Stop_ID	Route_Name	Region
R001	001	Mumbai-Pune	West
R001	002	Mumbai-Pune	West

Here, **Route_Name** and **Region** depend on **Route_ID** but not **Stop_ID**. To normalize:

- **Create two tables:**
 - **Routes** : Contains **Route_ID**, **Route_Name**, **Region**.
 - **Route_Stops** : Contains **Route_ID** and **Stop_ID**.

Normalized Structure:

1. Routes Table:

Route_ID	Route_Name	Region
R001	Mumbai-Pune	West

2. Route_Stops Table:

Route_ID	Stop_ID
R001	001
R001	002



Cheapest and shortest path webpage:

Algorithms Assumptions

- Floyd-Warshall Algorithm:
 - Assumed suitable for computing all-pairs in the network due to its ability to handle both positive weights (of distance and fares) and the relatively size of the data set.
- No Negative Cycle:
 - The network does not contain negative cycles, allowing the Floyd-Warshall algorithm to function correctly without adjustments for such cases.
- Currency:
 - All fares are amounts in the same currency to maintain uniformity during calculation.

User Interface

- Station Selection:
 - Users select start and end stations from dropdown menus populated with all available stations, ensuring valid input and enhancing user experience.
- Route Options:
 - Users can choose between finding the shortest route or cheapest route from a dropdown box menu.

Sample Data-set

Network Structure

The transit network consists of 12 stations connected through four main routes:

- 1.Route R1: Stations 1 → 2 → 3 → 4 → 5 → 6
- 2.Route R2: Stations 2 → 7 → 8 → 9 → 10 → 11 → 12
- 3.Route R3: Stations 1 → 3 → 5 → 7 → 9 → 12
- 4.Route R4: Stations 4 → 6 → 8 → 10 → 11

Example Paths Between Stations

- From Main Street Station (1) to Twelfth Avenue (12):
 - Option 1 (Via Route R3): 1 → 3 → 5 → 7 → 9 → 12
 - Option 2 (Via Routes R1 and R2): 1 → 2 → 7 → 8 → 9 → 10 → 11 → 12

Fare Prices

Fare Amounts per Route

- Route R1: 250 cents
- Route R2: 300 cents
- Route R3: 150 cents
- Route R4: 400 cents

Fare Calculation Examples

- Option 1 (Via Route R3):
 - Single fare of 150 cents (no transfers required).

- Option 2 (Via Routes R1 and R2):
 - Fare on Route R1: 250 cents
 - Fare on Route R2: 300 cents
 - Total Fare: $250 + 300 = 550$ cents

Working and Implementation Details

Algorithms Used

- Floyd-Warshall Algorithm: Implemented to compute the shortest paths between all pairs of stations in terms of both distance and fare.
 - Distance Matrix: Represents the distances between directly connected stations.
 - Fare Matrix: Represents the fares between directly connected stations.
- Path Reconstruction: Uses the next_node matrix generated by the Floyd-Warshall algorithm to reconstruct the optimal path between the selected start and end stations.

Data Structures

- Adjacency Matrices:
 - Distance Matrix (distance_matrix): A 2D list where distance_matrix[i][j] holds the distance from station i to station j.
 - Fare Matrix (fare_matrix): A 2D list where fare_matrix[i][j] holds the fare from station i to station j.
- Station Index Mapping (stop_index): A dictionary mapping station IDs to their indices in the adjacency matrices.

Application Workflow

1. Data Retrieval:
 - Fetch all station IDs and names from the database.
 - Build the distance_matrix and fare_matrix using data from stop_distances and fare information.
2. User Input:
 - Users select start and end stations and choose either the shortest or cheapest route option.
3. Algorithm Execution:
 - The Floyd-Warshall algorithm computes the shortest paths based on the selected criterion (distance or fare).
 - The algorithm updates the dist and next_node matrices accordingly.
4. Path Reconstruction:
 - The application reconstructs the optimal path from the next_node matrix.
 - It converts station indices back to station IDs and retrieves station names for display.
5. Result Presentation:
 - Displays the computed path, including the list of stations and the total distance or fare.
 - If no path is found, informs the user accordingly.

The complete code along with the sample data has been uploaded on [Github](#)

Step - 1

Transit System

Plan Your Trip

Start Point:
Main Street Station

End Point:
Eighth Avenue

Route Option:
Shortest

Find Route

Step - 2

Transit System

Plan Your Trip

Start Point:
Main Street Station

End Point:
Sixth Avenue

Main Street Station
Second Avenue Stop
Third Street Stop
Fourth Avenue
Fifth Street
Sixth Avenue
Seventh Street
Eighth Avenue
Ninth Street
Tenth Avenue
Eleventh Street
Twelfth Avenue

Step - 3

Transit System

Plan Your Trip

Start Point:
Main Street Station

End Point:
Sixth Avenue

Route Option:
Shortest

Shortest
Cheapest

Result

Transit System

Route Results

Total Distance (meters)	Stops
558	- Main Street Station - Third Street Stop - Fourth Avenue - Sixth Avenue

Back to Route Planner

© Transit System

Shortest distance

Cheapest path

Transit System

Route Results

Total Fare (cents)	Stops
550	- Main Street Station - Third Street Stop - Fifth Street - Sixth Avenue

Back to Route Planner

© Transit System

Inventory Management System for Spares on tyres

Objectives

- Ensure that spare tires are always available to avoid downtime.
- Minimize total costs (ordering, holding, and stockout costs).
- Track inventory levels, orders, and usage efficiently.

Assumptions

- The average lifespan of a tire is 50,000 km.
- The replacement rate for tires is proportional to the mileage of buses.
- Inventory is managed periodically, with orders placed when stock falls below a certain threshold.
- Costs include:
 - Ordering cost: Fixed cost for ordering a batch of tires.
 - Holding cost: Cost of storing tires in the inventory.
 - Stockout cost: Penalty for running out of stock.

The model that can be used to solve our problem are given below:

EOQ MODEL

$$EOQ = \sqrt{\frac{2DS}{H}}$$

where

- D =Annual demand (number of tires needed per year)
- S = Ordering cost per order
- H = Holding cost per tire per year

Purpose: To find the optimal number of units to order per batch so that inventory costs are minimized.

ROP MODEL

The reorder point is determined using :

$$ROP=d \times L$$

Where:

- d = Average daily demand
- L = Lead time (time between ordering and receiving tires)

For safety, add a safety stock (SS) to handle demand variability:

$$ROP=d \times L + SS$$

Purpose: Ensures sufficient stock is available to meet demand during the lead time.

Inventory Management

Current Inventory

Tire ID	Type	Quantity	Reorder Point	Supplier ID
---------	------	----------	---------------	-------------

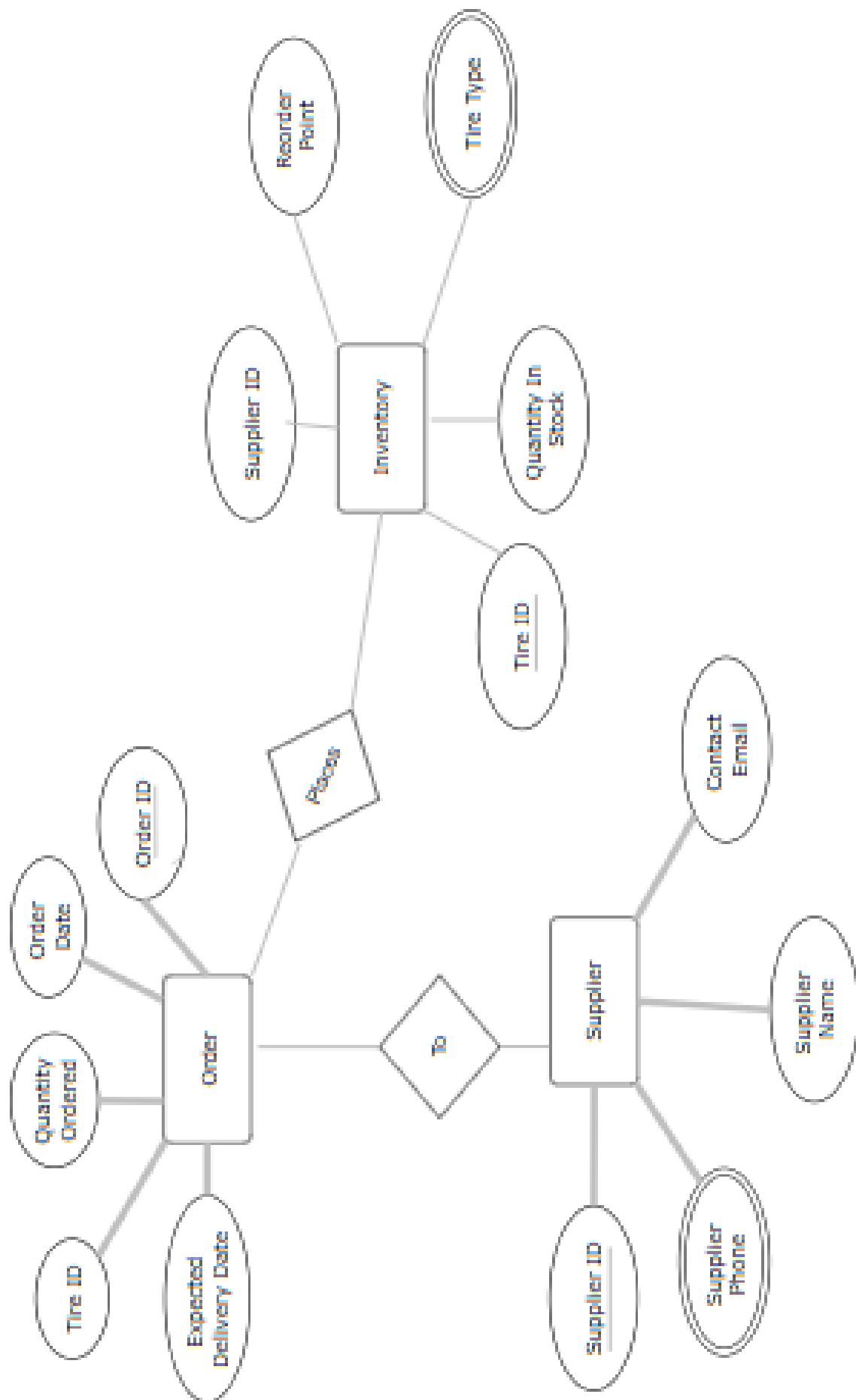
Add a New Tire

MRF	2	24	1	Add Tire
-----	---	----	---	----------

Place an Order

Tire ID	Quantity	21/11/2024	29/11/2024	Place Order
---------	----------	------------	------------	-------------

E-R Diagram of Inventory Model



Important links to the codes:

Transport database and path finder UI-

<https://github.com/Gsparsh22/DMS624-Transit-System>

Inventory management system-

<https://github.com/nagmani004/DMS624-Project>

SBI Loan application UI-

https://github.com/Vatsal-Rajput/SBI_Loan_UI

The END!!!